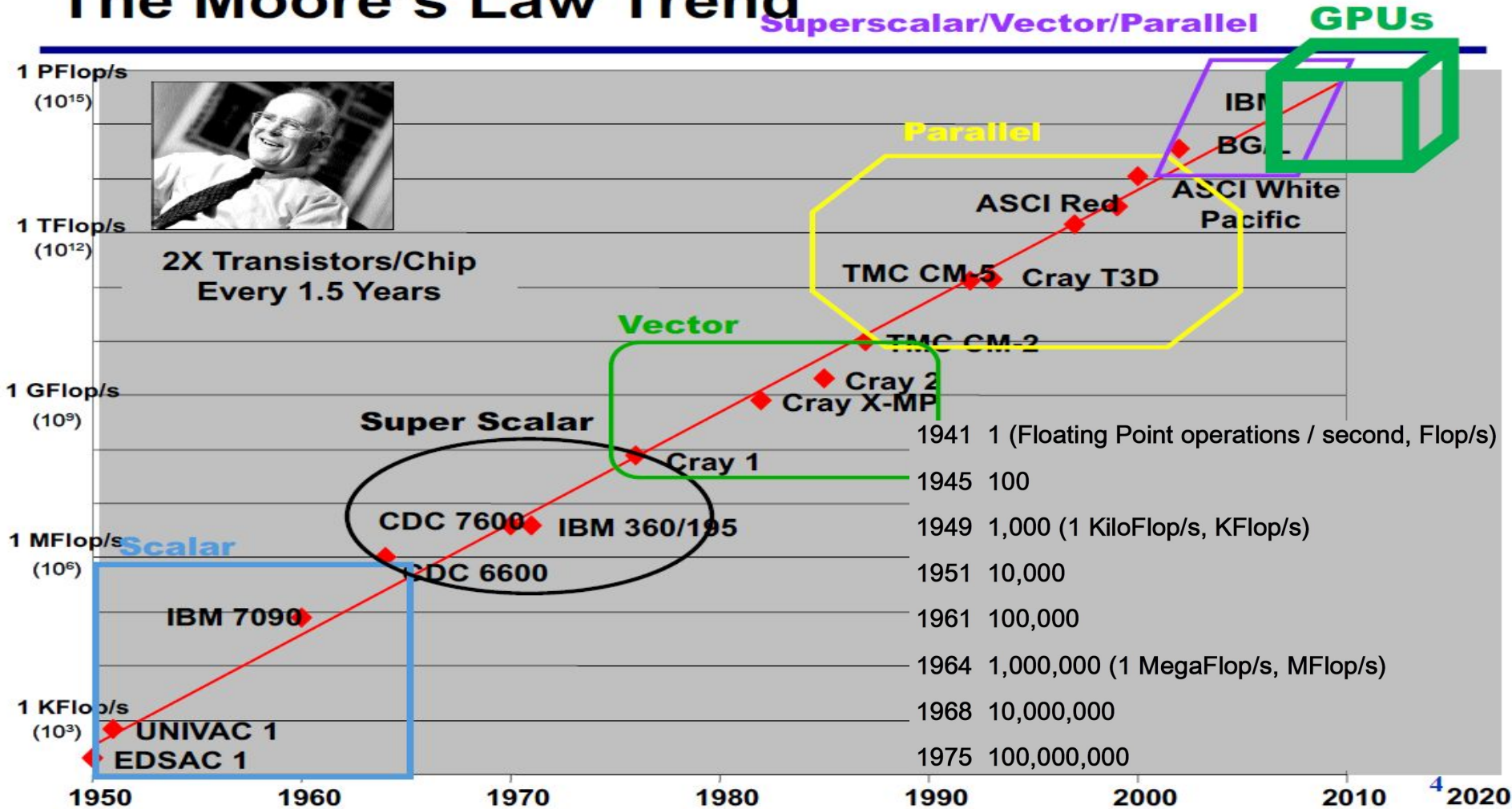
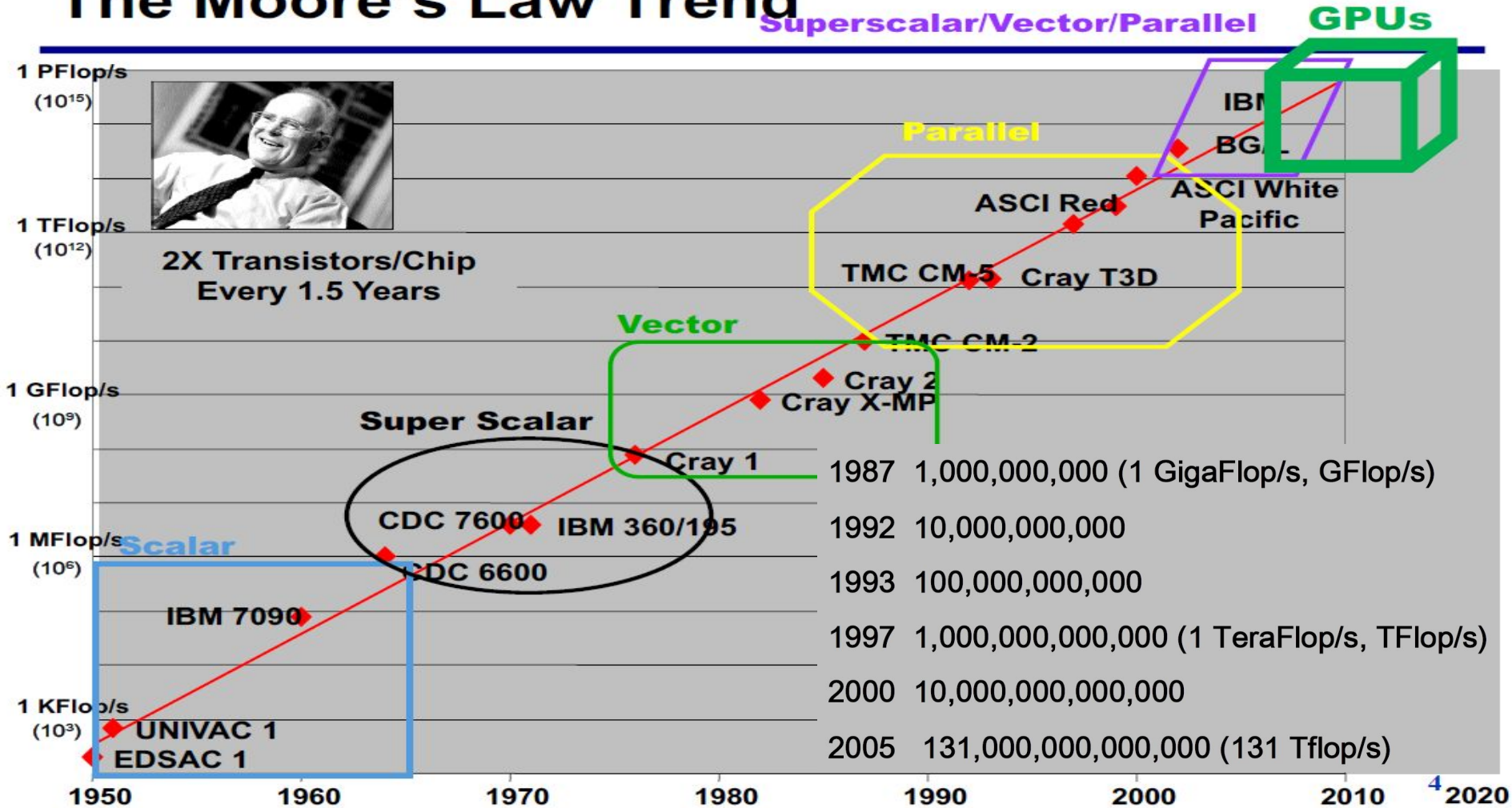


Domain Specific Architectures

The Moore's Law Trend



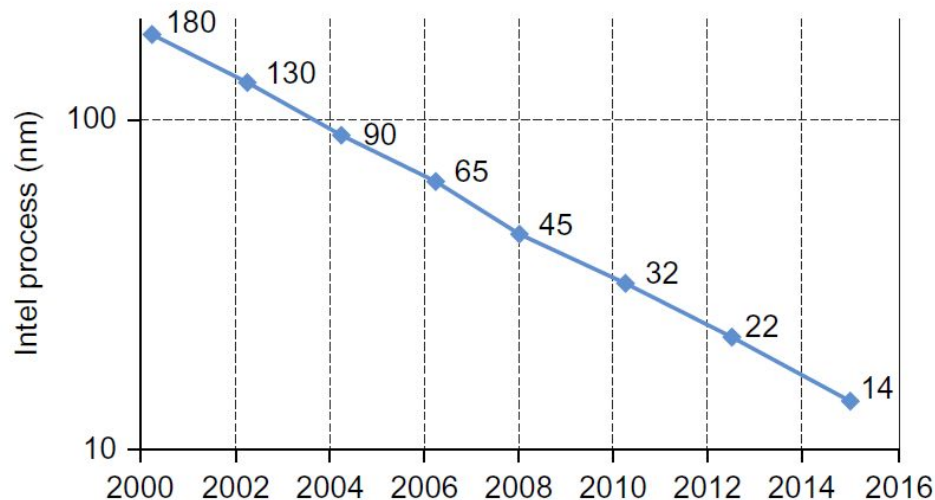
The Moore's Law Trend



Moore's Law trend

Moore's Law enabled:

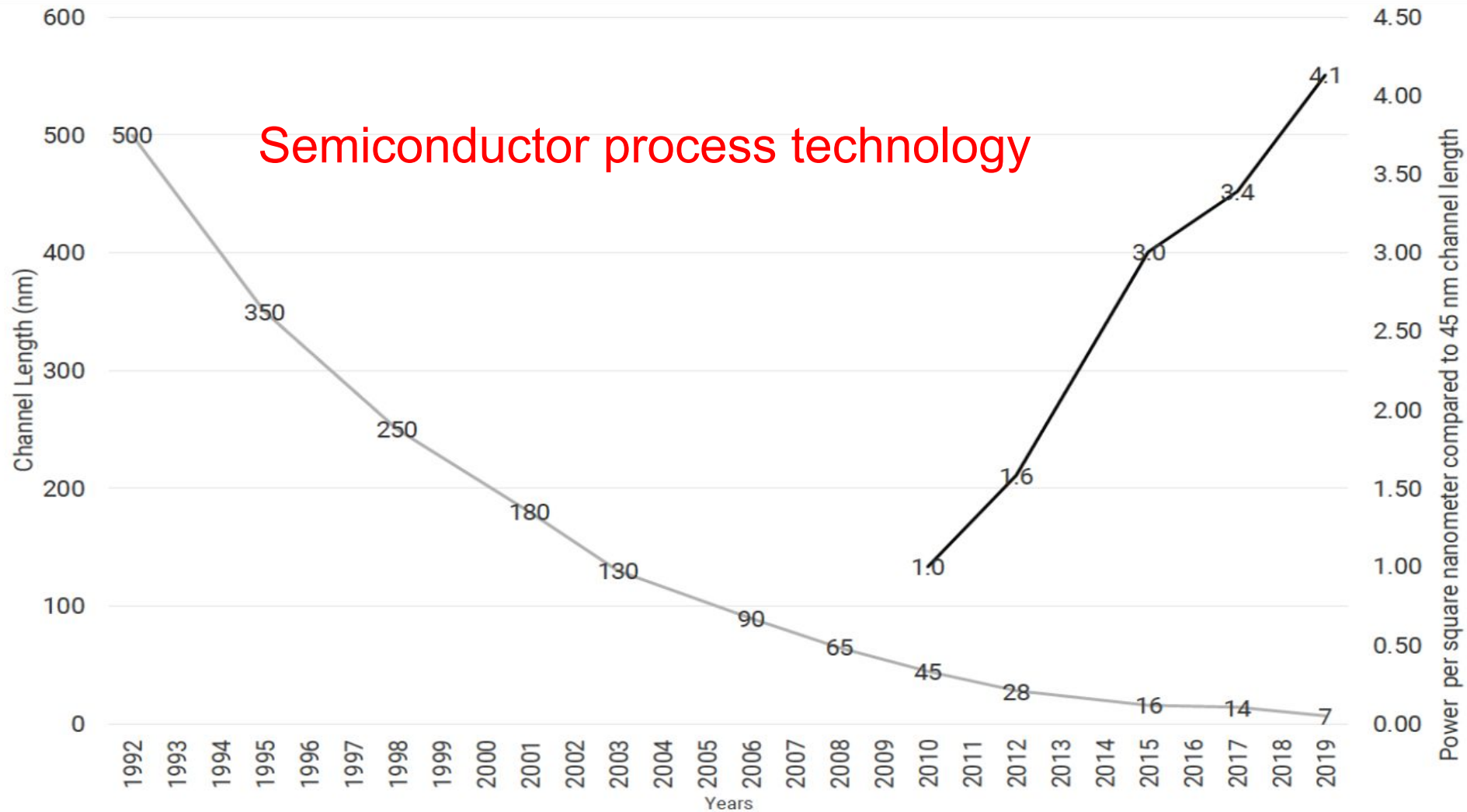
- Deep memory hierarchy, e.g. 3- or even 4-level caches
- Wide SIMD units, e.g. 512 bit SIMD register
- Deep pipelines, e.g. 10-20 stages
- Branch prediction, e.g. close to > 90% accurate rate
- Out-of-order execution to achieve data-flow and remove
- Speculative prefetching
- Multithreading
- Multiprocessing



The End of Traditional Scaling

- Dennard Scaling has ended — transistor power density no longer decreases with size.
- Moore's Law is slowing — transistor doubling no longer yields proportional performance gains.
- Consequence: General-purpose architectures face diminishing returns in performance, cost, and energy efficiency.

Semiconductor process technology



Limits of Traditional Performance Growth

- Implementation technologies (like smaller process nodes) are near physical limits.
- Parallelism — once the main path to higher performance — now faces scaling bottlenecks.
- Increasing cores or threads gives less benefit per added resource.

The Shift Toward Domain Specific Architectures (DSAs)

- DSAs propose a new paradigm for performance enhancement.
- They focus on hardware acceleration for specific computational domains.
- Belief: Major performance leaps now require domain-specific optimization, not generic architectures.

Why DSAs Matter

- DSAs can dramatically improve performance for well-defined domains.
- They offer energy efficiency by eliminating unnecessary general-purpose functionality.
- They align hardware capabilities directly with application characteristics.

RISC instruction		Overhead	ALU	125 pJ
Load/Store	D-\$	Overhead	ALU	150 pJ
SP floating point			+	15–20 pJ
32-bit addition			+	7 pJ
8-bit addition			+	0.2–0.5 pJ

Figure 7.2 Energy costs in picoJoules for a 90 nm process to fetch instructions or access a data cache compared to the energy cost of arithmetic operations (Qadeer et al., 2015).

Remembering Amdahl's Law

- Amdahl's Law: The overall speedup from acceleration is limited by the fraction of time spent in the accelerated portion.
- Implication: DSAs must target large portions of computation to meaningfully improve total performance.
- Small accelerators for niche tasks yield minimal runtime benefit.

Power and Cost as Dominant Design Drivers

- In large-scale computing (e.g., data centers), **power consumption** dominates cost and deployment decisions.
- DSAs offer power-efficient computation, aligning with **energy and thermal constraints**.
- The **warehouse-scale computer** environment amplifies these concerns — efficiency directly affects operational cost.

The Future of Performance Innovation

- The era of “free” performance gains from transistor scaling is over.
- **Architectural specialization** is now the main frontier for improvement.
- DSAs will coexist with general-purpose CPUs, forming **heterogeneous systems** tailored to workload demands.

NRE Cost Comparison: ASICs, FPGAs, and DSAs

- NRE refers to the one-time engineering cost to design, develop, and test new hardware.
- Includes design effort, tooling, verification, mask generation, and fabrication setup.
- Once paid, NRE does not recur for each produced unit — it's a fixed upfront investment.
- Critical factor when choosing between ASICs, FPGAs, and DSAs.

NRE in ASICs

- Highest NRE cost among hardware options.
- Involves full custom circuit design, mask creation, and fabrication setup.
- Provides maximum performance and energy efficiency for a fixed function.
- Economically viable only for high-volume production where costs can be amortized.
- Modern ASIC tape-out can exceed tens of millions of dollars in NRE.

NRE in FPGAs

- Very low NRE cost — no fabrication or mask costs; reconfigurable hardware.
- Uses pre-fabricated logic blocks and programmable interconnects.
- Ideal for prototyping, low-volume, or rapidly evolving applications.
- Trade-offs: Lower performance and higher power consumption compared to ASICs.

NRE in DSAs

- DSAs often fall between ASICs and FPGAs in NRE cost.
- Built for specific domains (e.g., AI, graphics, data analytics).
- NRE cost depends on design reuse, domain maturity, and manufacturing scale.
- Benefit: Balance between specialization (like ASIC) and reuse (like FPGA).

ASICs vs. FPGAs vs. DSAs — NRE Trade-offs

Feature / Cost Factor	ASICs	FPGAs	DSAs
NRE Cost	Very High	Very Low	Moderate
Performance	Highest	Lowest	High (domain-optimized)
Energy Efficiency	Excellent	Poor–Moderate	High
Flexibility	None (fixed)	Full (reconfig.)	Limited (domain-specific)
Per-Unit Cost (at scale)	Lowest	Highest	Moderate
Time to Market	Long	Short	Medium

Guidelines for Domain Specific Architectures (DSAs)

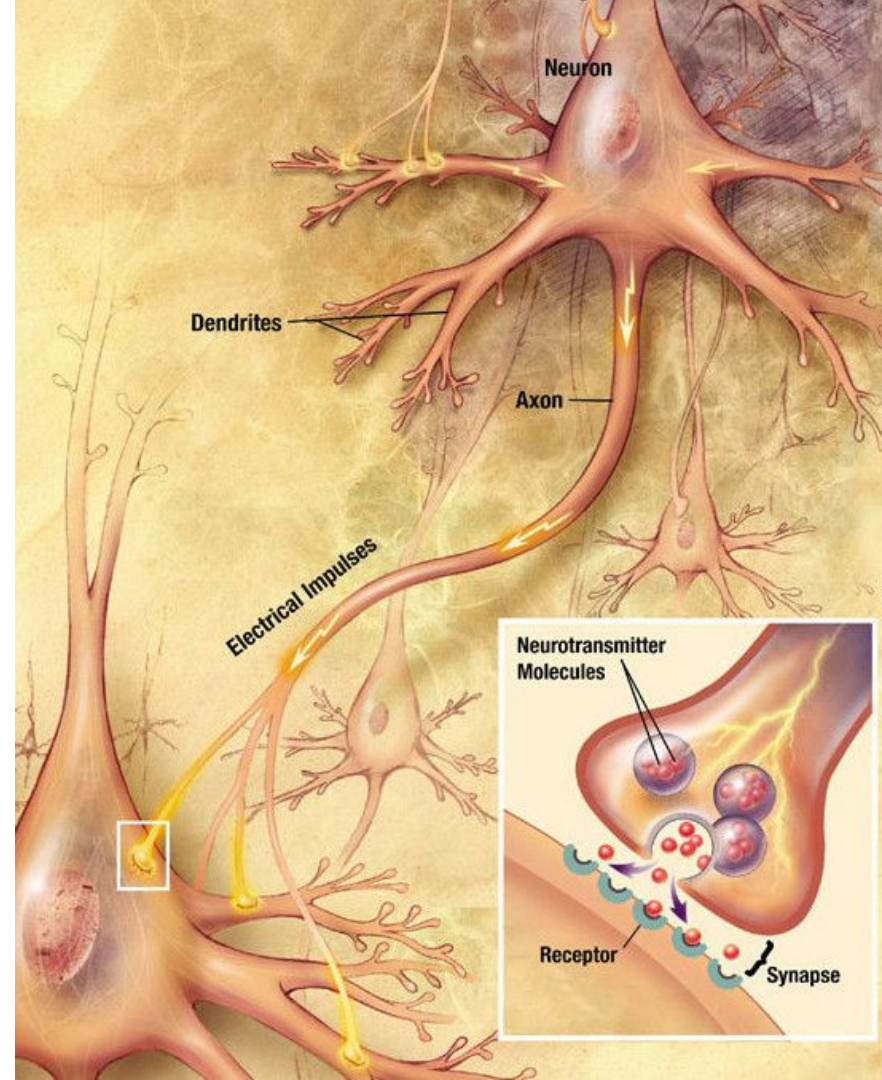
1. Use dedicated memories to minimize distances of data movement
 - a. Hardware-controlled multi-level cache
 - b. domain-specific software controlled scratch-pad
2. Invest resources into more arithmetic units or bigger memories
 - a. Core optimization (speculation, threading, etc), more domain-specific FU/memory
3. Use the easiest form of parallelism that matches the domain
 - a. MIMD, SIMD or VLIW that matches domain
4. Reduce data size and type to the simplest needed for the domain
 - a. General-purpose 32/64 integer/float; domain-specific 8/16 int/float
5. Use a domain-specific programming language to port code to the DSA
 - a. General-purpose C/C++/Fortran
 - b. Domain-specific language: Halide for vision processing, TensorFlow for DNN

Guideline	TPU	Catapult	Crest	Pixel Visual Core
Design target	Data center ASIC	Data center FPGA	Data center ASIC	PMD ASIC/SOC IP
1. Dedicated memories	24 MiB Unified Buffer, 4 MiB Accumulators	Varies	N.A.	Per core: 128 KiB line buffer, 64 KiB P.E. memory
2. Larger arithmetic unit	65,536 Multiply-accumulators	Varies	N.A.	Per core: 256 Multiply-accumulators (512 ALUs)
3. Easy parallelism	Single-threaded, SIMD, in-order	SIMD, MISD	N.A.	MPMD, SIMD, VLIW
4. Smaller data size	8-Bit, 16-bit integer	8-Bit, 16-bit integer 32-bit Fl. Pt.	21-bit Fl. Pt.	8-bit, 16-bit, 32-bit integer
5. Domain-specific lang.	TensorFlow	Verilog	TensorFlow	Halide/TensorFlow

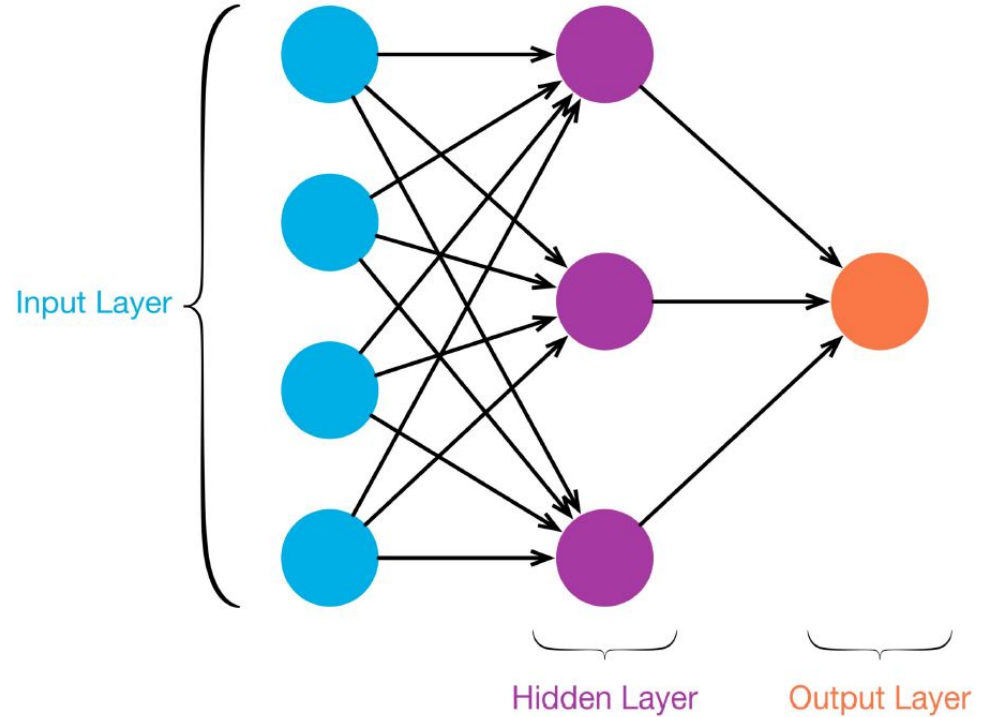
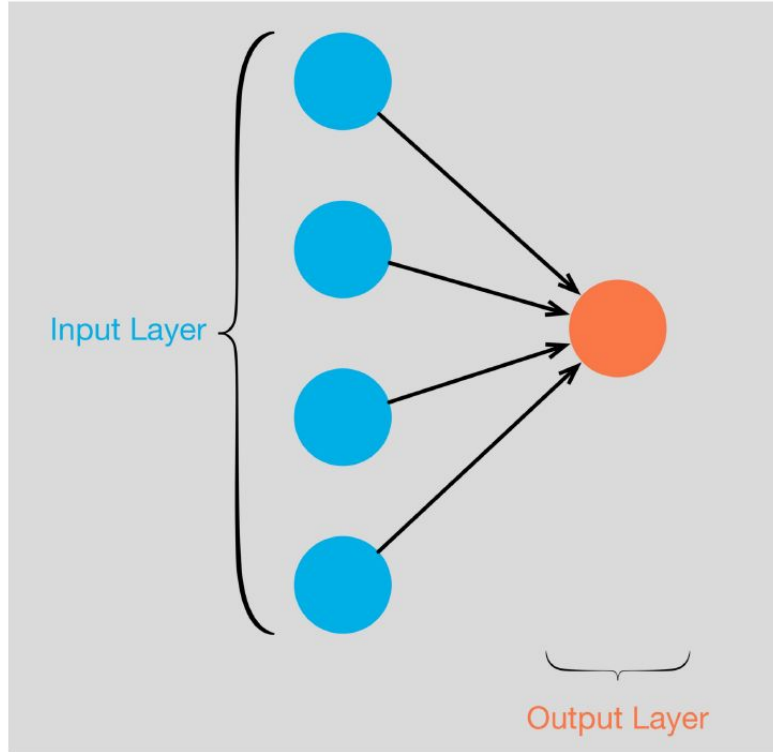
Figure 7.3 The four DSAs in this chapter and how closely they followed the five guidelines. Pixel Visual Core typically has 2–16 cores. The first implementation of Pixel Visual Core does not support 8-bit arithmetic.

Example: Deep Neural Network

- A Deep Neural Network (DNN) is a computational model inspired by the human brain's structure and function.
- It consists of multiple layers of interconnected nodes (neurons).
- Each neuron applies a mathematical transformation to its input and passes the result to the next layer.
- “Deep” refers to having multiple hidden layers between input and output.

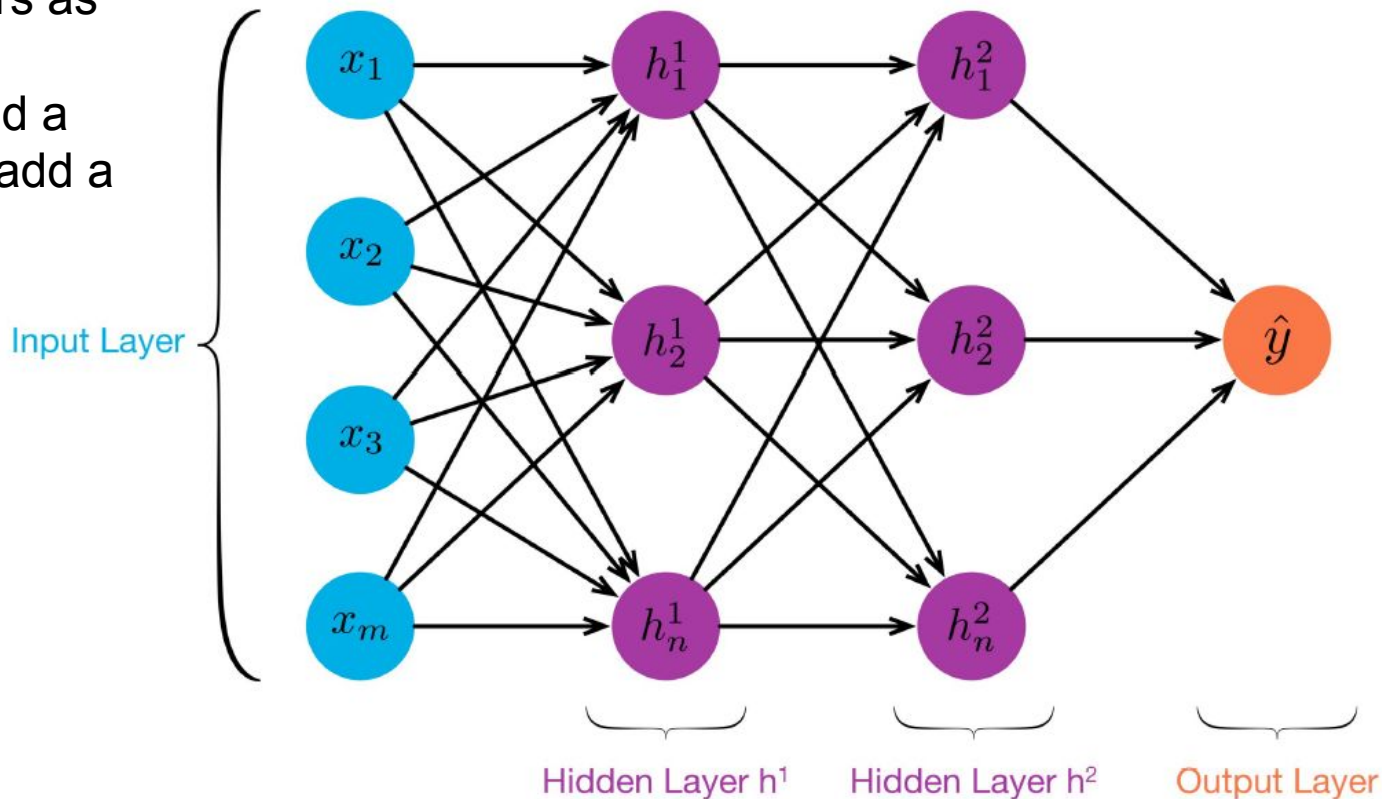


Multi-Layer Network



<https://towardsdatascience.com/multi-layer-neural-networks-with-sigmoid-function-deep-learning-for-rookies-2-bf464f09eb7f>

- You can introduce as many hidden layers as you want.
- Every time you add a hidden layer, you add a set of weights.



- Single-layer:

$$h = \mathbf{w}^T \mathbf{x}$$

- Hidden-layer:

$$h = \mathbf{W}^T \mathbf{x}$$

- Two Hidden Layers:

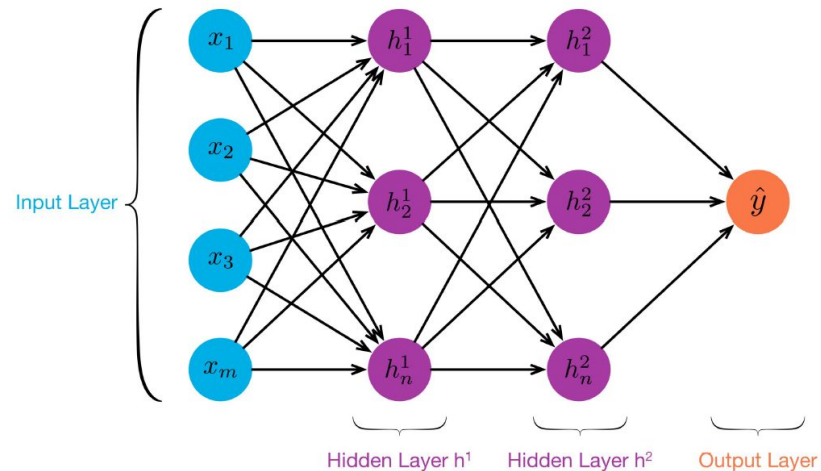
$$h = \mathbf{W}_2^T \mathbf{W}_1^T \mathbf{x}$$

- Three Hidden Layers:

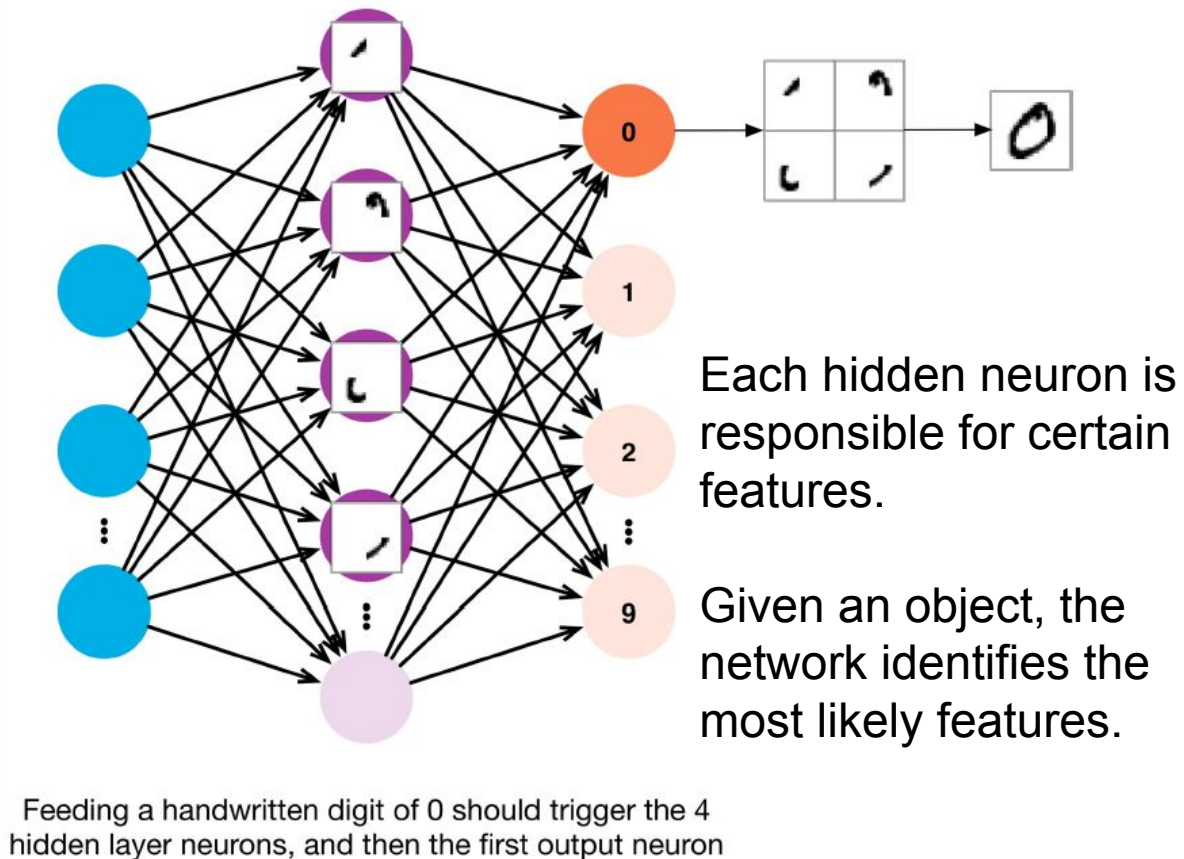
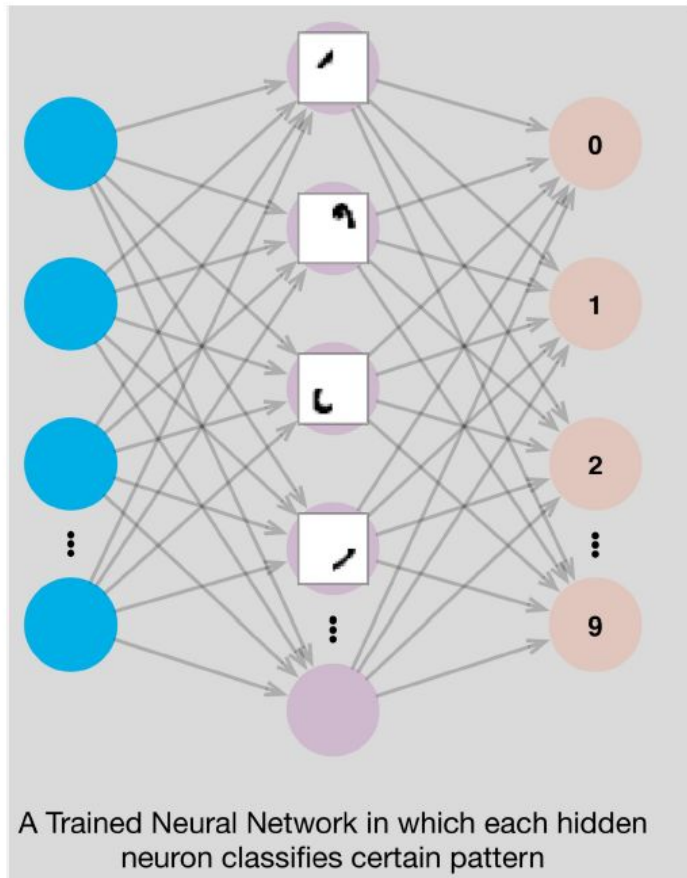
$$h = \mathbf{W}_3^T \mathbf{W}_2^T \mathbf{W}_1^T \mathbf{x}$$

- A LOT of Hidden Layers:

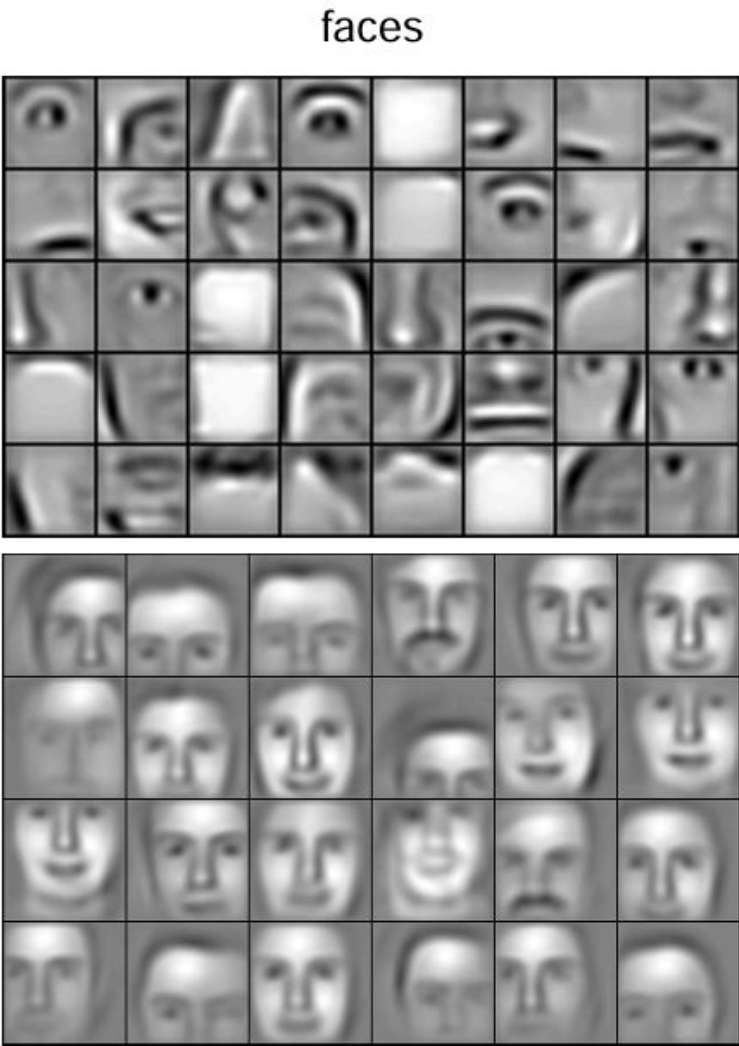
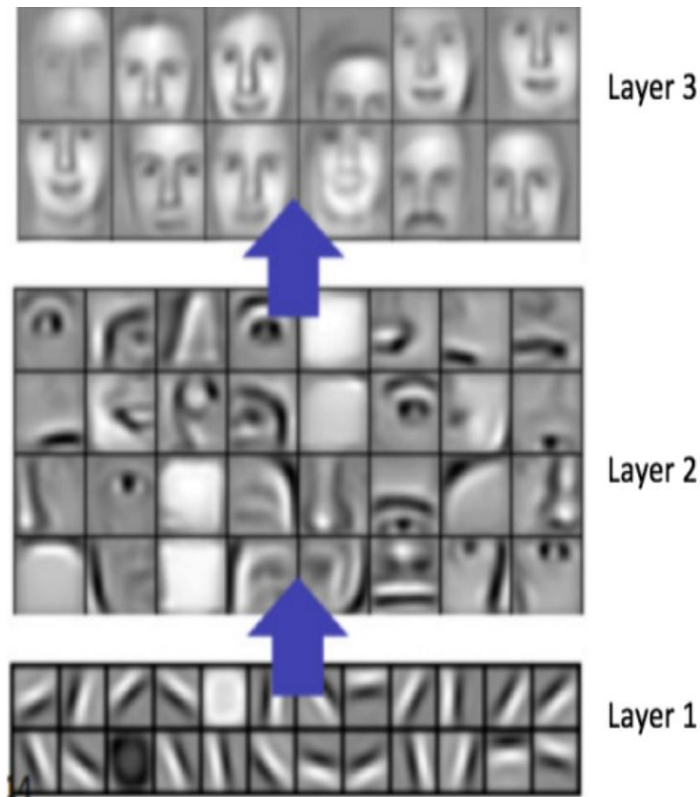
$$h = \mathbf{W}_N^T \dots \mathbf{W}_2^T \mathbf{W}_1^T \mathbf{x}$$



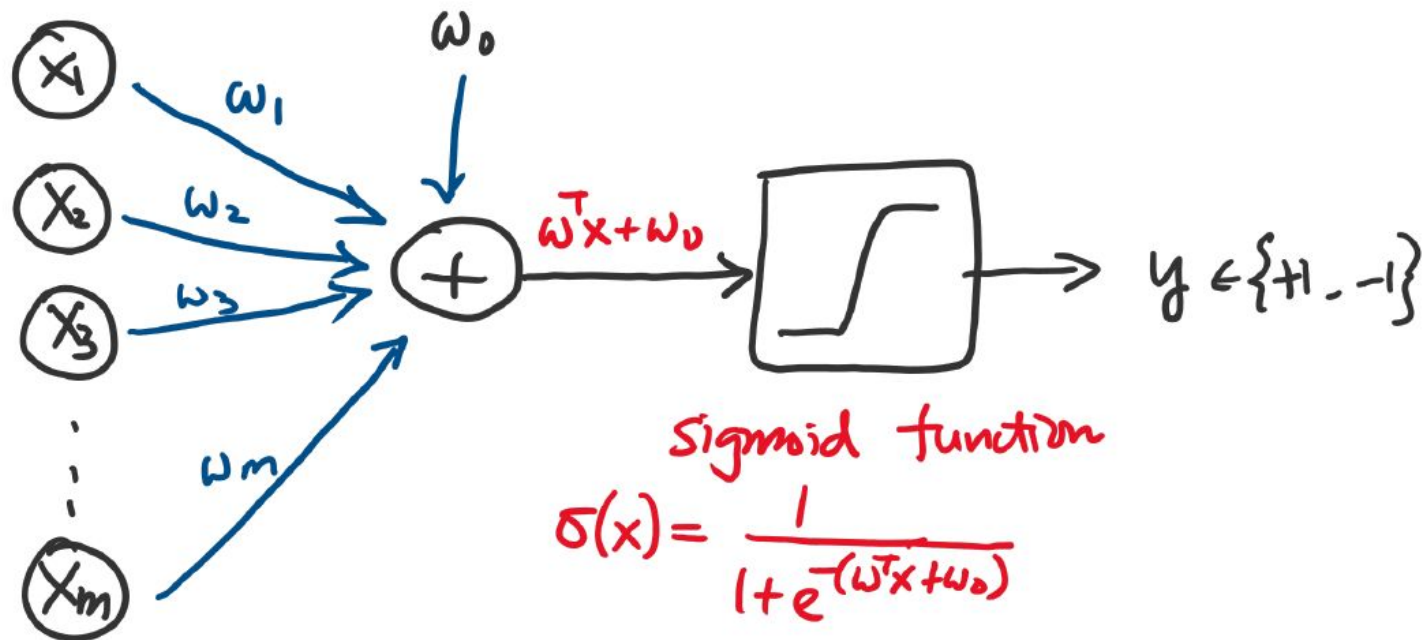
Interpreting the Hidden Layer



Interpreting the Hidden Layer contd...



Single-Layer Perceptron



- Input neurons $\mathbf{x}$
- Weights $\mathbf{w}$
- Predicted label $= \sigma(\mathbf{w}^T \mathbf{x} + w_0)$.

$$\begin{bmatrix} h_1^1 \\ h_2^1 \\ \vdots \\ h_n^1 \end{bmatrix} = \begin{bmatrix} w_{11}^1 & w_{12}^1 & \dots & w_{1n}^1 \\ w_{21}^1 & w_{22}^1 & \dots & w_{2n}^1 \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}^1 & w_{m2}^1 & \dots & w_{mn}^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

How do we efficiently learn the weights?

- Here is the loss function you want to minimize:

$$J(\mathbf{w}_1, \dots, \mathbf{w}_L) = \sum_{i=1}^N \|\sigma(\mathbf{w}_L^T \dots \sigma(\mathbf{w}_2^T \sigma(\mathbf{w}_1^T \mathbf{x}_i))) - \mathbf{y}_i\|^2$$

- You have a set of nonlinear activation functions, usually the sigmoid.
- To optimize, you need gradient descent. For example, for $\mathbf{w}_1$

$$\mathbf{w}_1^{t+1} = \mathbf{w}_1^t - \alpha \nabla J(\mathbf{w}_1^t)$$

- But you need to do this for all $\mathbf{w}_1, \dots, \mathbf{w}_L$.
- And there are lots of sigmoid functions.
- Let us do the brute force.
- And this is back-propagation. (Really? Yes...)

- Single-layer:

$$h = \mathbf{w}^T \mathbf{x}$$

- Hidden-layer:

$$h = \mathbf{W}^T \mathbf{x}$$

- Two Hidden Layers:

$$h = \mathbf{w}_2^T \mathbf{w}_1^T \mathbf{x}$$

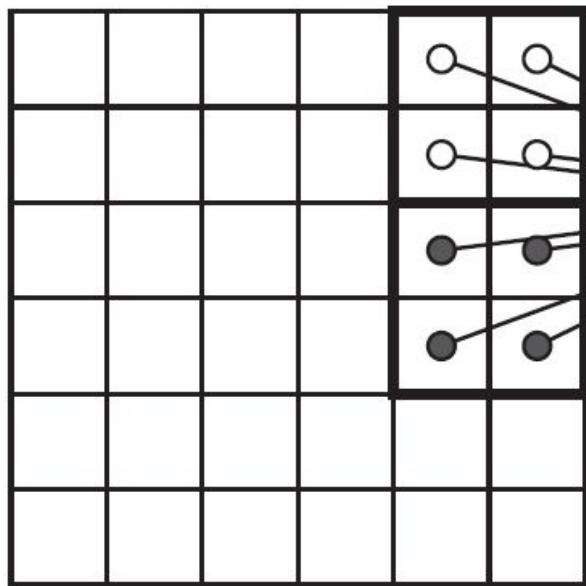
- Three Hidden Layers:

$$h = \mathbf{w}_3^T \mathbf{w}_2^T \mathbf{w}_1^T \mathbf{x}$$

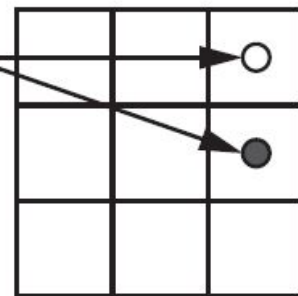
- A LOT of Hidden Layers:

$$h = \mathbf{w}_N^T \dots \mathbf{w}_2^T \mathbf{w}_1^T \mathbf{x}$$

Input image

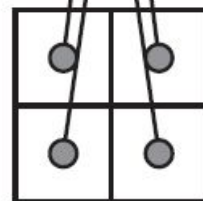


Output feature map



VMX

nlf



Weights



Vector matrix multiply



Nonlinear function

Convolutional Neural Network

1	0	1	0	1	0
0	1	1	0	1	1
1	0	1	0	1	0
1	0	1	1	1	0
0	1	1	0	1	1
1	0	1	0	1	0

Input



1	0	1
0	1	1
1	0	1

Image patch
(Local receptive field)

*

1	2	3
4	5	6
7	8	9

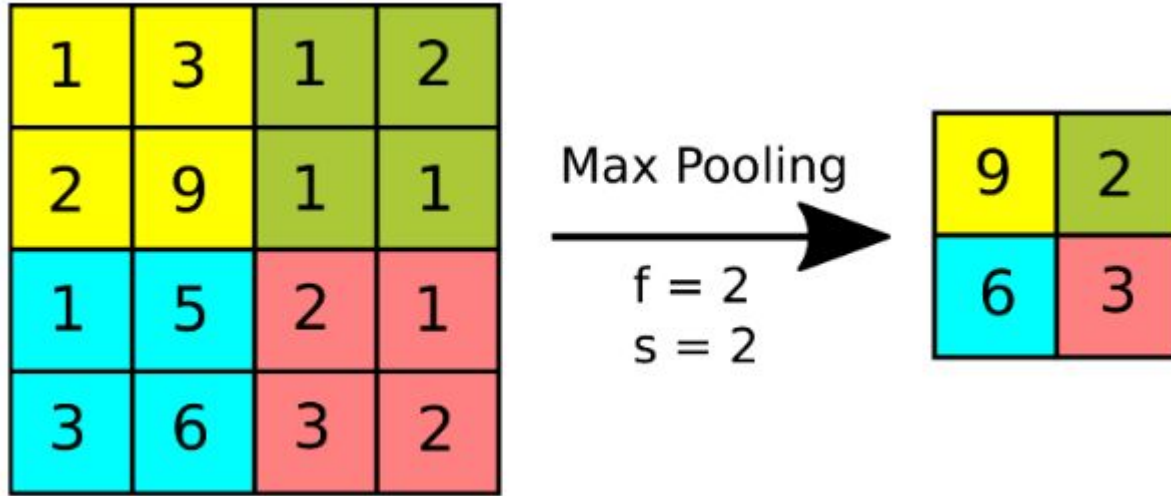
Kernel
(filter)



31			

Output

Max pooling layer



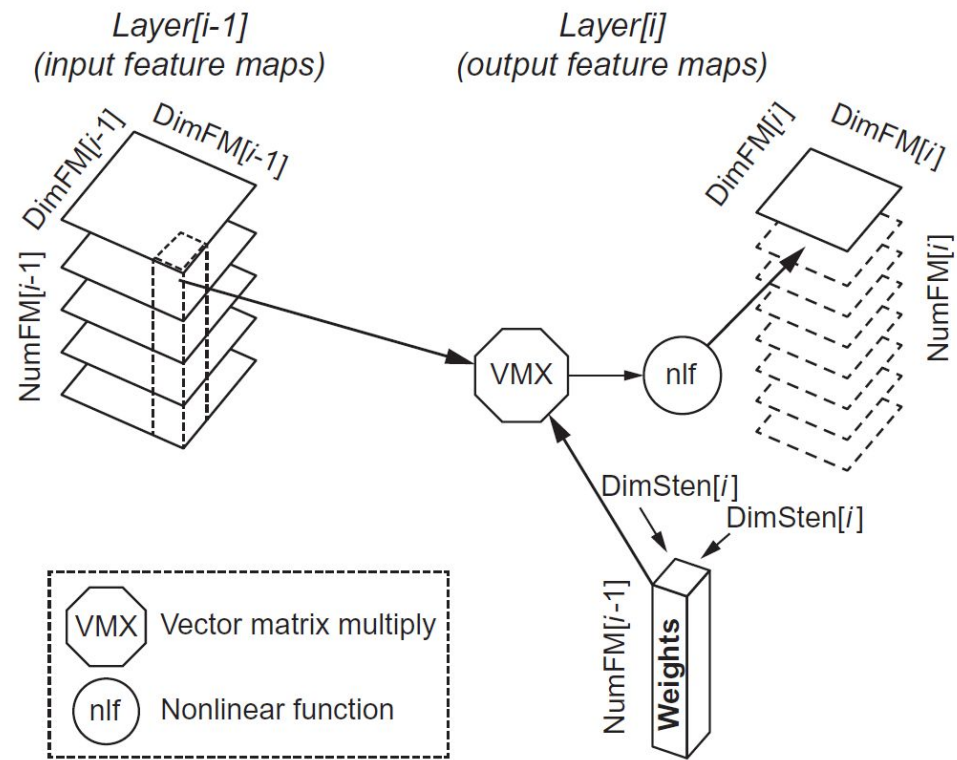
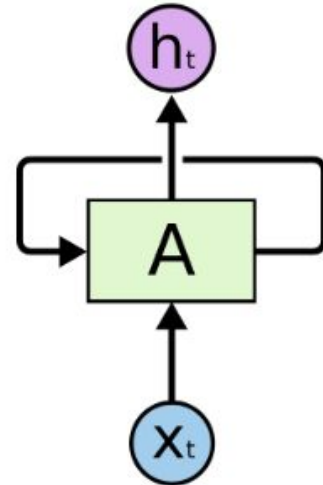


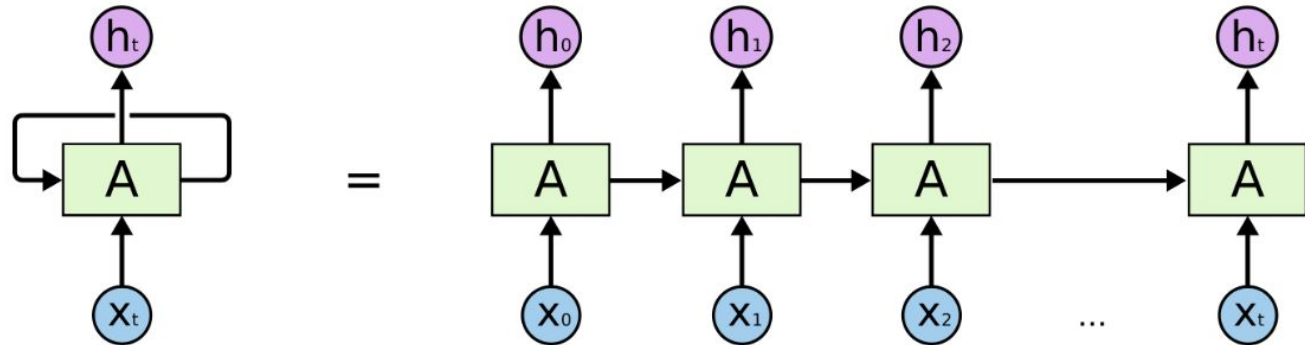
Figure 7.9 CNN general step showing input feature maps of Layer[i-1] on the left, the output feature maps of Layer[i] on the right, and a three-dimensional stencil over input feature maps to produce a single output feature map. Each output feature map has its own unique set of weights, and the vector-matrix multiply happens for every one. The dotted lines show future output feature maps in this figure. As this figure illustrates, the dimensions and number of the input and output feature maps are often different. As with MLPs, ReLU is a popular nonlinear function for CNNs.

Recurrent Neural Network

- A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data, where the order of the inputs matters.
- Unlike standard (feedforward) neural networks, RNNs have feedback connections that allow information to persist across time steps.
- The network receives one input at a time: example, one word in a sentence
- It updates an internal hidden state that stores information about previous steps.
- This hidden state is reused at the next time step, forming a recurrent loop.

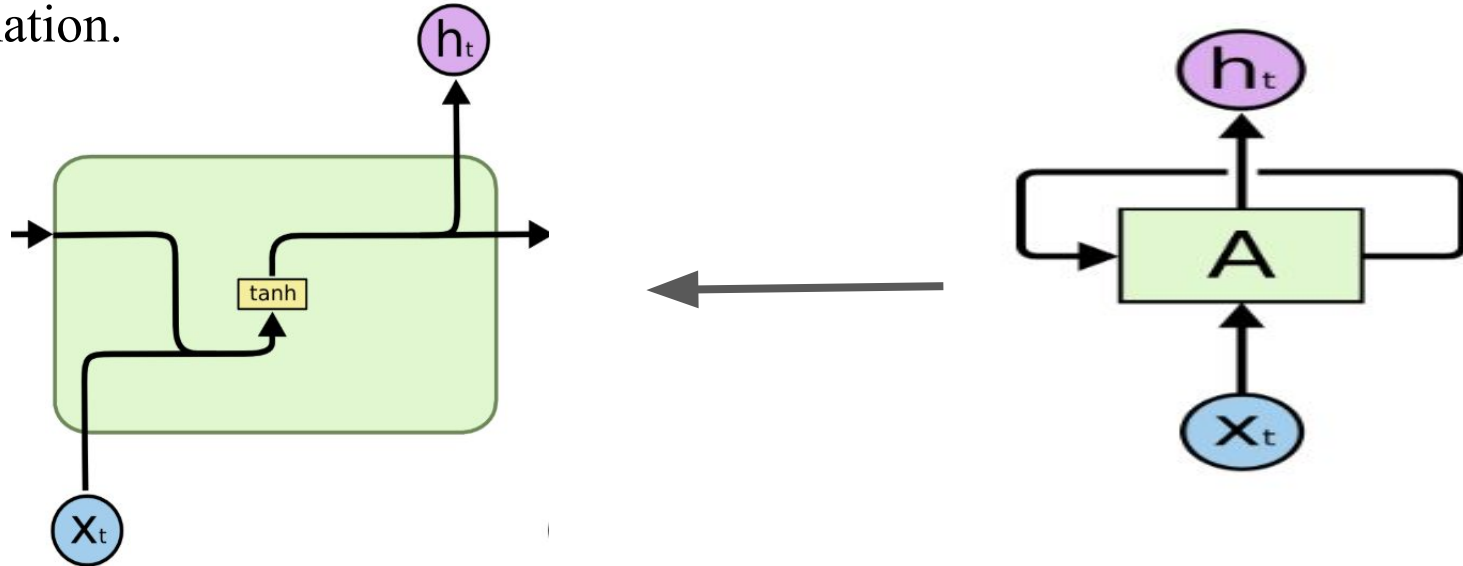


- A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor.
- RNNs shown incredible success to a variety of problems: speech recognition, language modeling, translation, image captioning etc..
- For example, consider a language model trying to predict the next word based on the previous ones. If we are trying to predict the last word in “the clouds are in the sky,” we don’t need any further context – it’s pretty obvious the next word is going to be sky. In such cases, where the gap between the relevant information and the place that it’s needed is small, RNNs can learn to use the past information.

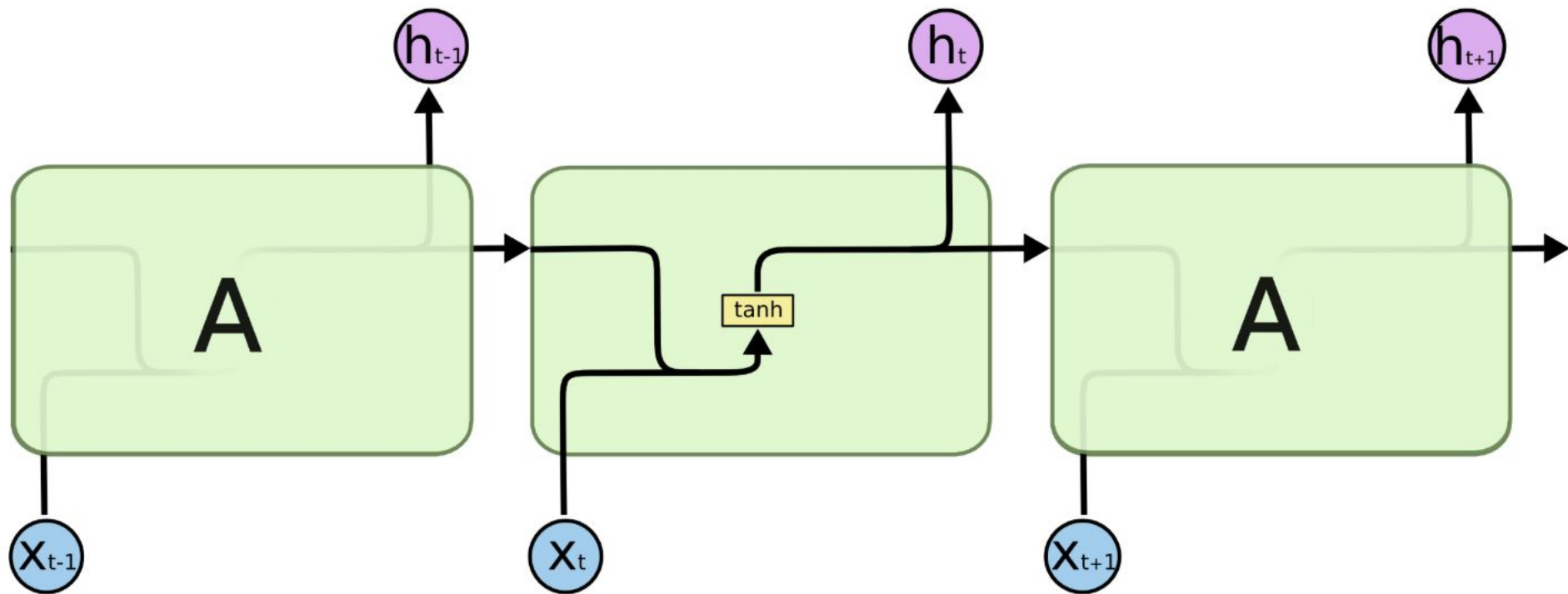


An unrolled recurrent neural network.

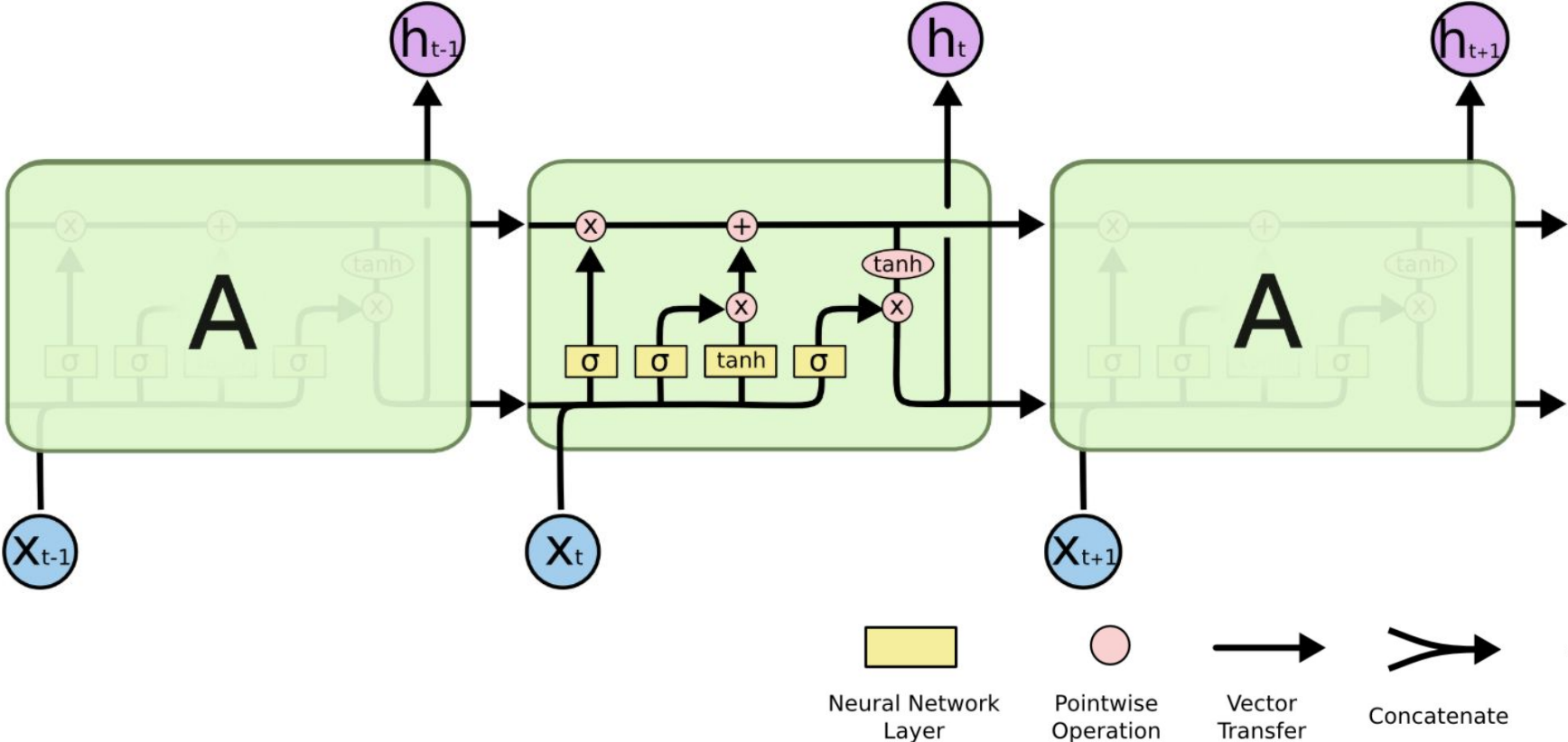
- But there are also cases where we need more context. Consider trying to predict the last word in the text “I grew up in France... I speak fluent French.”
- Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back. It’s entirely possible for the gap between the relevant information and the point where it is needed to become very large.
- Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.



RNN model

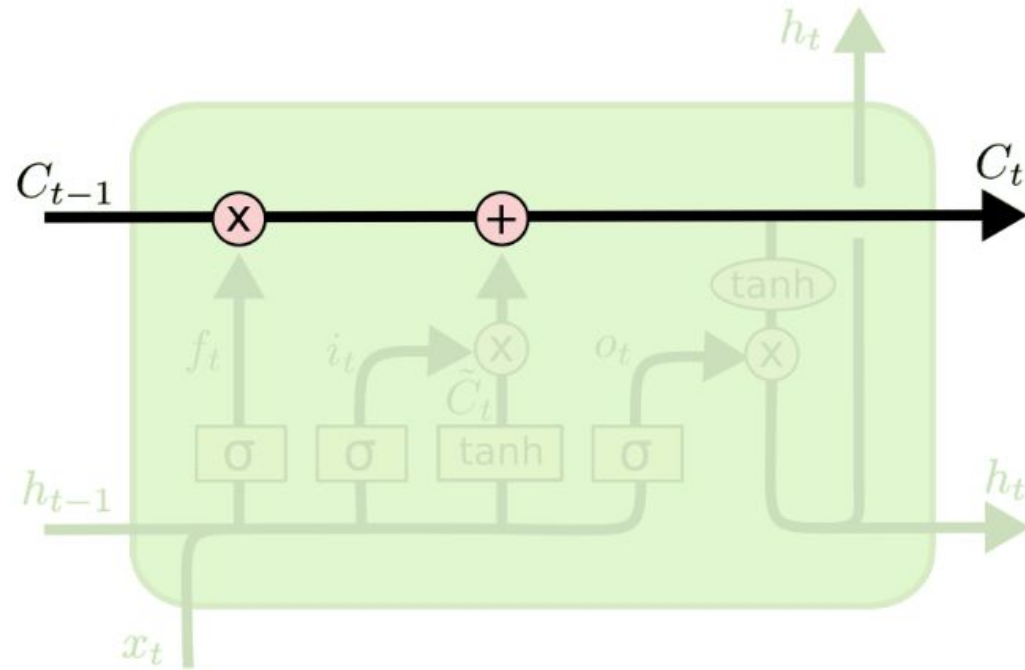


LSTM cell



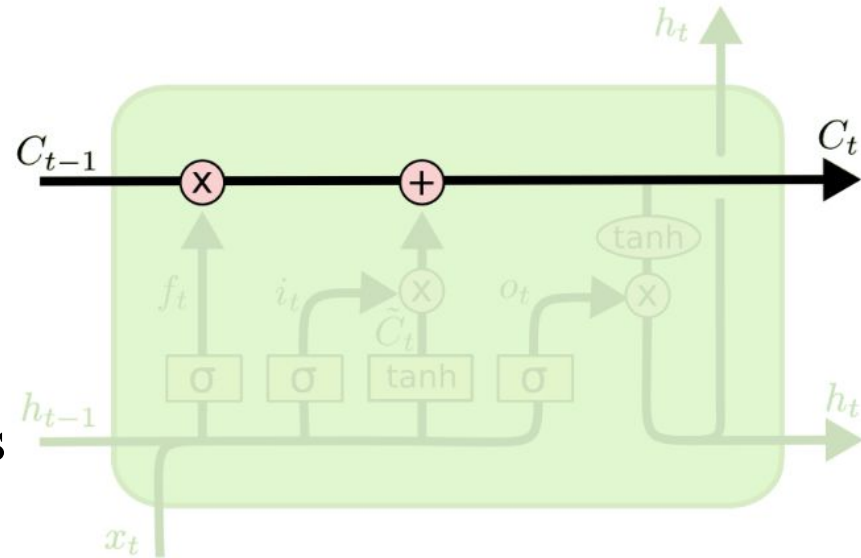
LSTM contd..

Long Short Term Memory networks – usually just called “LSTMs” – are a special kind of RNN, capable of learning long-term dependencies. They were introduced by Hochreiter & Schmidhuber (1997).



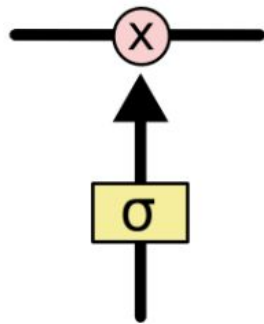
LSTM contd..

- The key to LSTMs is the cell state, the horizontal line running through the top of the diagram.
- The cell state is kind of like a conveyor belt.
- It runs straight down the entire chain, with only some minor linear interactions.
- It's very easy for information to just flow along it unchanged.
- The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called gates.



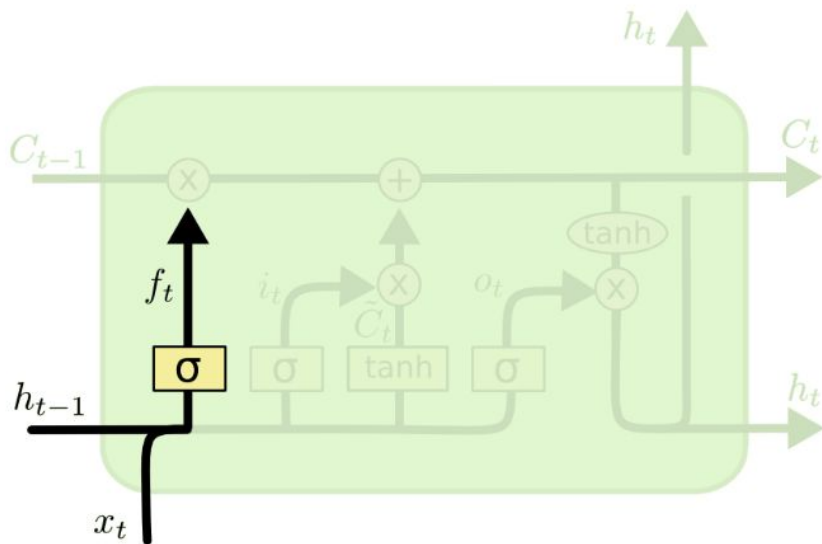
Gates in LSTM cell

- Gates are a way to optionally let information through. They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.
- The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through. A value of zero means “let nothing through,” while a value of one means “let everything through!”



Forget gate

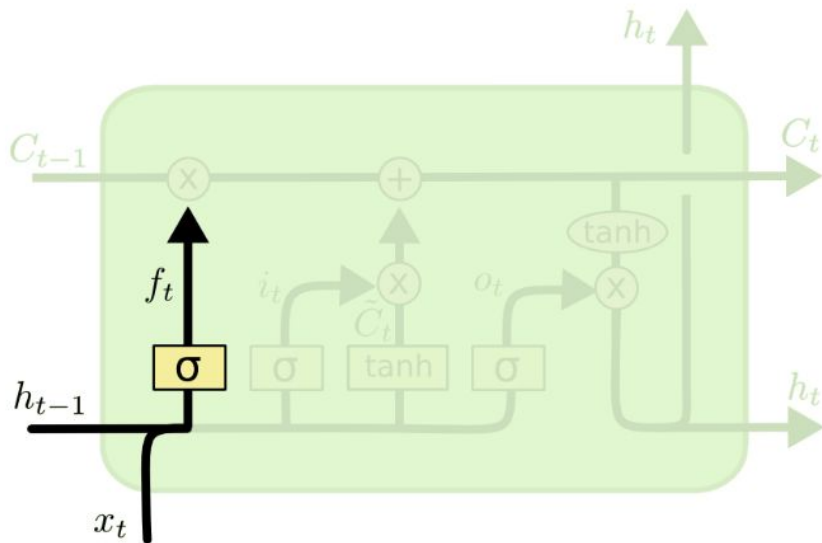
- The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the **“forget gate layer.”**
- It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} .



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Forget gate

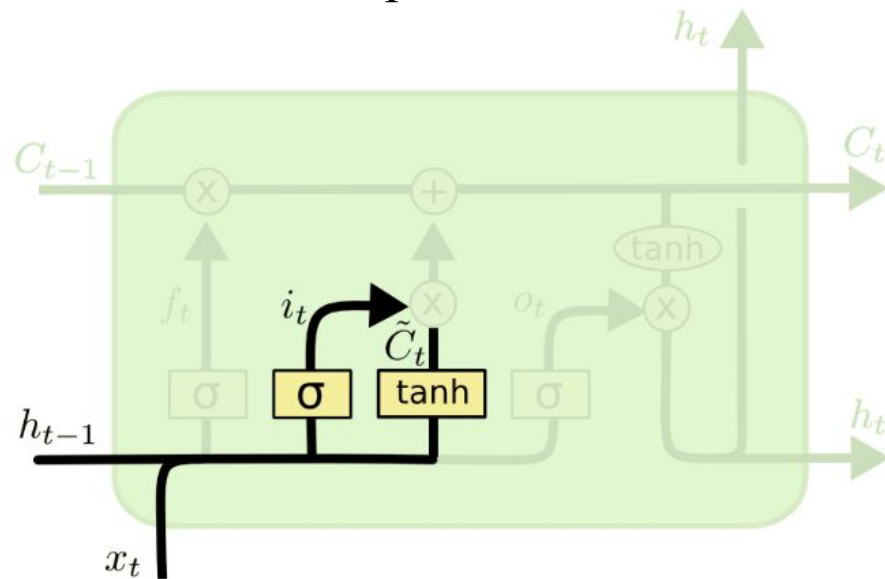
- Gate output '1' represents “completely keep this” while a '0' represents “completely get rid of this.”
- *For language translation:* The cell state might include the gender of the present subject, so that the correct pronouns can be used. When we see a new subject, we want to forget the gender of the old subject.



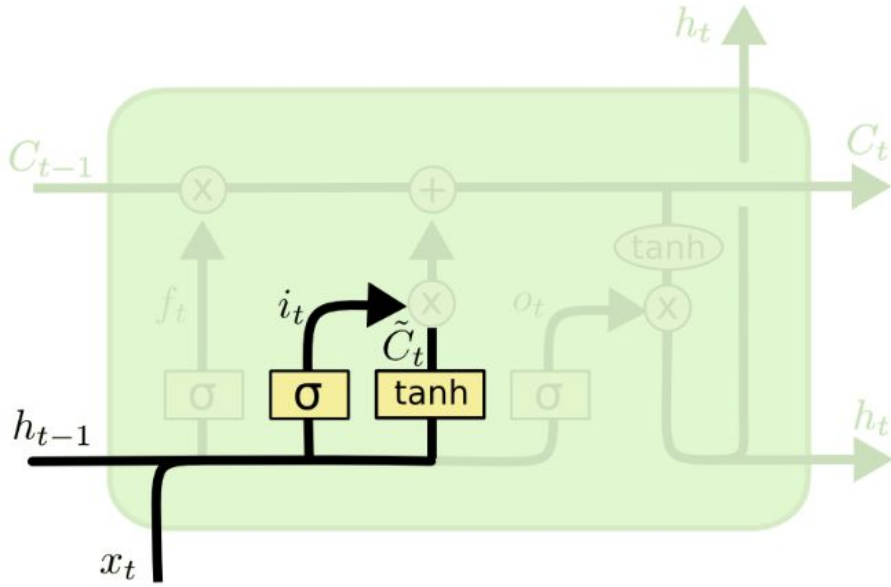
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate

- The next step is to decide what new information we're going to store in the cell state.
- This has two parts. First, a sigmoid layer called the “input gate layer” decides which values we'll update. Next, a tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state. In the next step, we'll combine these two to create an update to the state.
- In the example of our language model, we'd want to add the gender of the new subject to the cell state, to replace the old one we're forgetting.



Input Gate



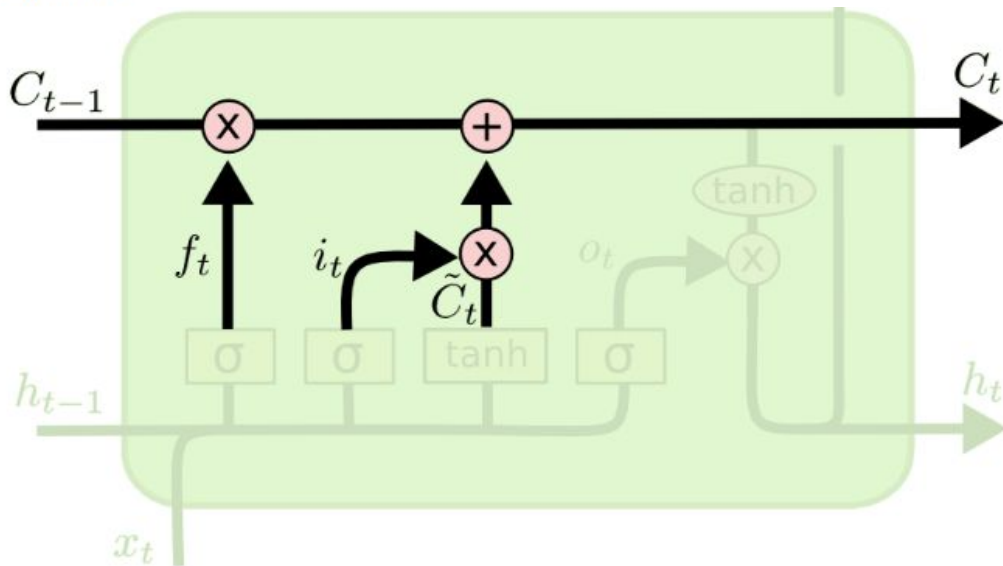
$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Update Cell State

- Now it's now time to update the old cell state, C_{t-1} , into the new cell state C_t .
- The previous steps already decided what to do, we just need to actually do it.

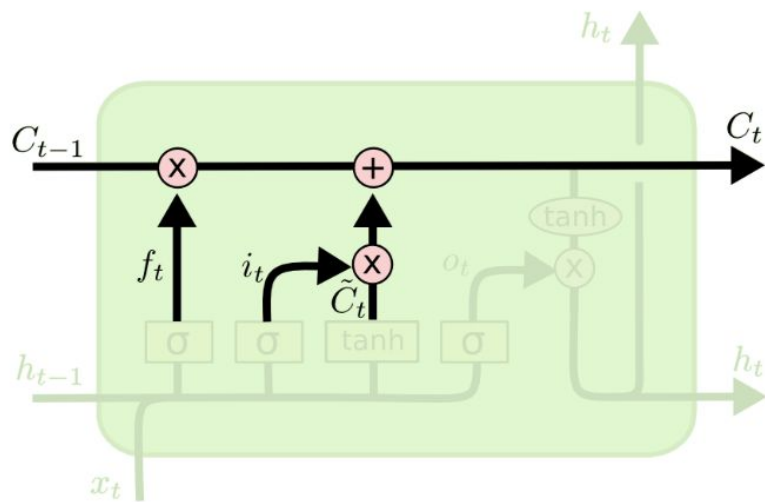
We multiply the old state by f_t , forgetting the things we decided to forget earlier. Then we add $i_t * \tilde{C}_t$. This is the new candidate values, scaled by how much we decided to update each state value.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- In the case of the language model, this is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in the previous steps.

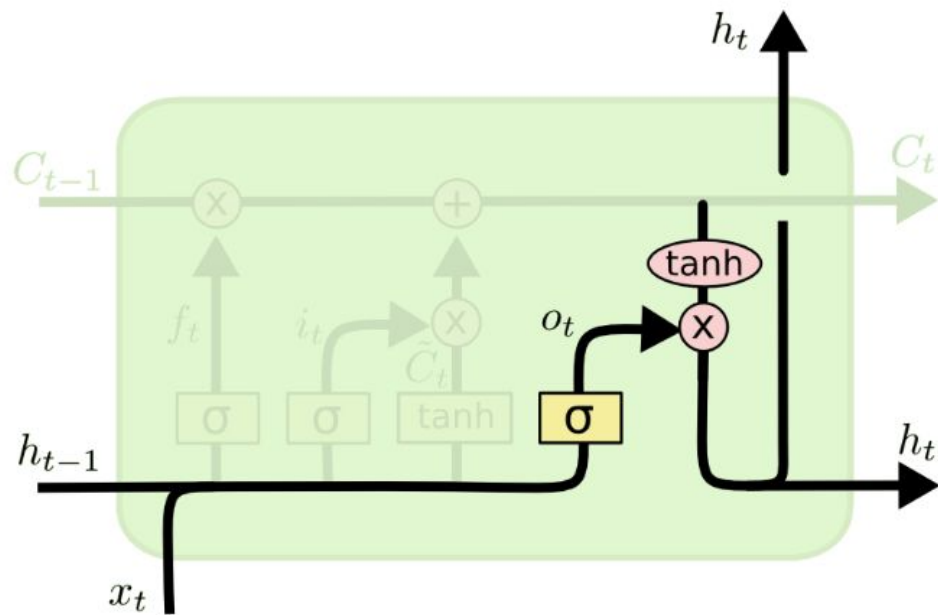
We multiply the old state by f_t , forgetting the things we decided to forget earlier. Then we add $i_t * \tilde{C}_t$. This is the new candidate values, scaled by how much we decided to update each state value.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

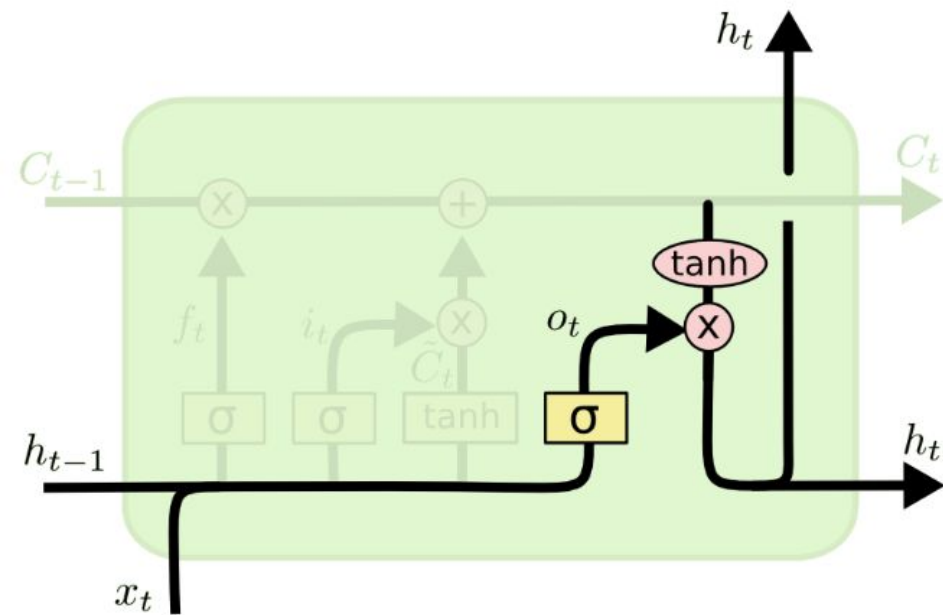
Output Gate

- Finally, we need to decide what we're going to output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output.
- Then, we put the cell state through *tanh* (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.



Output Gate

- For the language model example, since it just saw a subject, it might want to output information relevant to a verb, in case that's what is coming next. For example, it might output whether the subject is singular or plural, so that we know what form a verb should be conjugated into if that's what follows next.



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Name	DNN layers	Weights	Operations/Weight
MLP0	5	20M	200
MLP1	4	5M	168
LSTM0	58	52M	64
LSTM1	56	34M	96
CNN0	16	8M	2888
CNN1	89	100M	1750

Figure 7.5 Six DNN applications that represent 95% of DNN workloads for inference at Google in 2016, which we use in [Section 7.9](#). The columns are the DNN name, the number of layers in the DNN, the number of weights, and operations per weight (operational intensity). [Figure 7.41](#) on page 595 goes into more detail on these DNNs.

Training Versus Inference

Type of data	Problem area	Size of benchmark's training set	DNN architecture	Hardware	Training time
text [1]	Word prediction (word2vec)	100 billion words (Wikipedia)	2-layer skip gram	1 NVIDIA Titan X GPU	6.2 hours
audio [2]	Speech recognition	2000 hours (Fisher Corpus)	11-layer RNN	1 NVIDIA K1200 GPU	3.5 days
images [3]	Image classification	1 million images (ImageNet)	22-layer CNN	1 NVIDIA K20 GPU	3 weeks
video [4]	activity recognition	1 million videos (Sports-1M)	8-layer CNN	10 NVIDIA GPUs	1 month

Figure 7.6 Training set sizes and training time for several DNNs ([landola, 2016](#)).

Google TPU

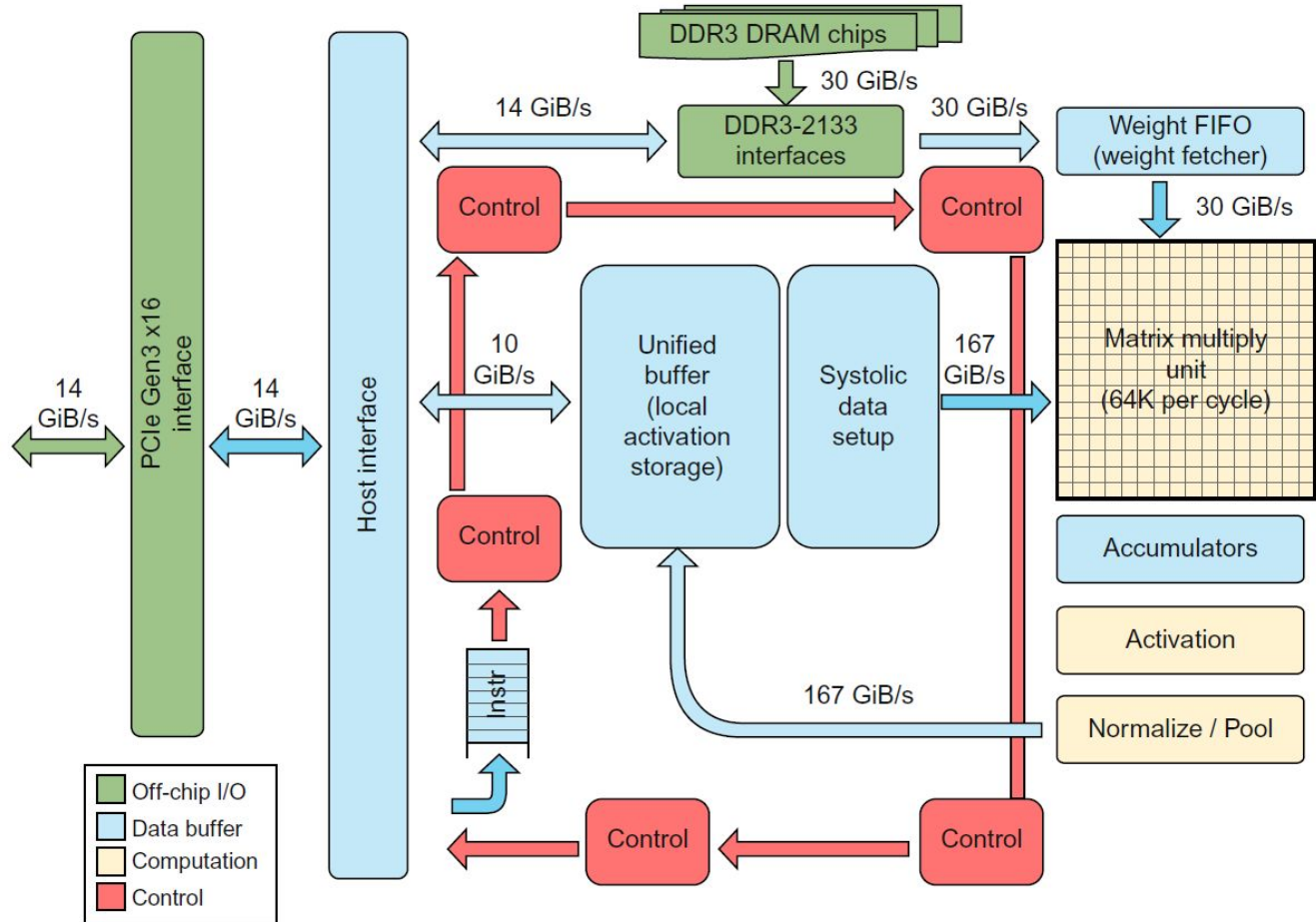
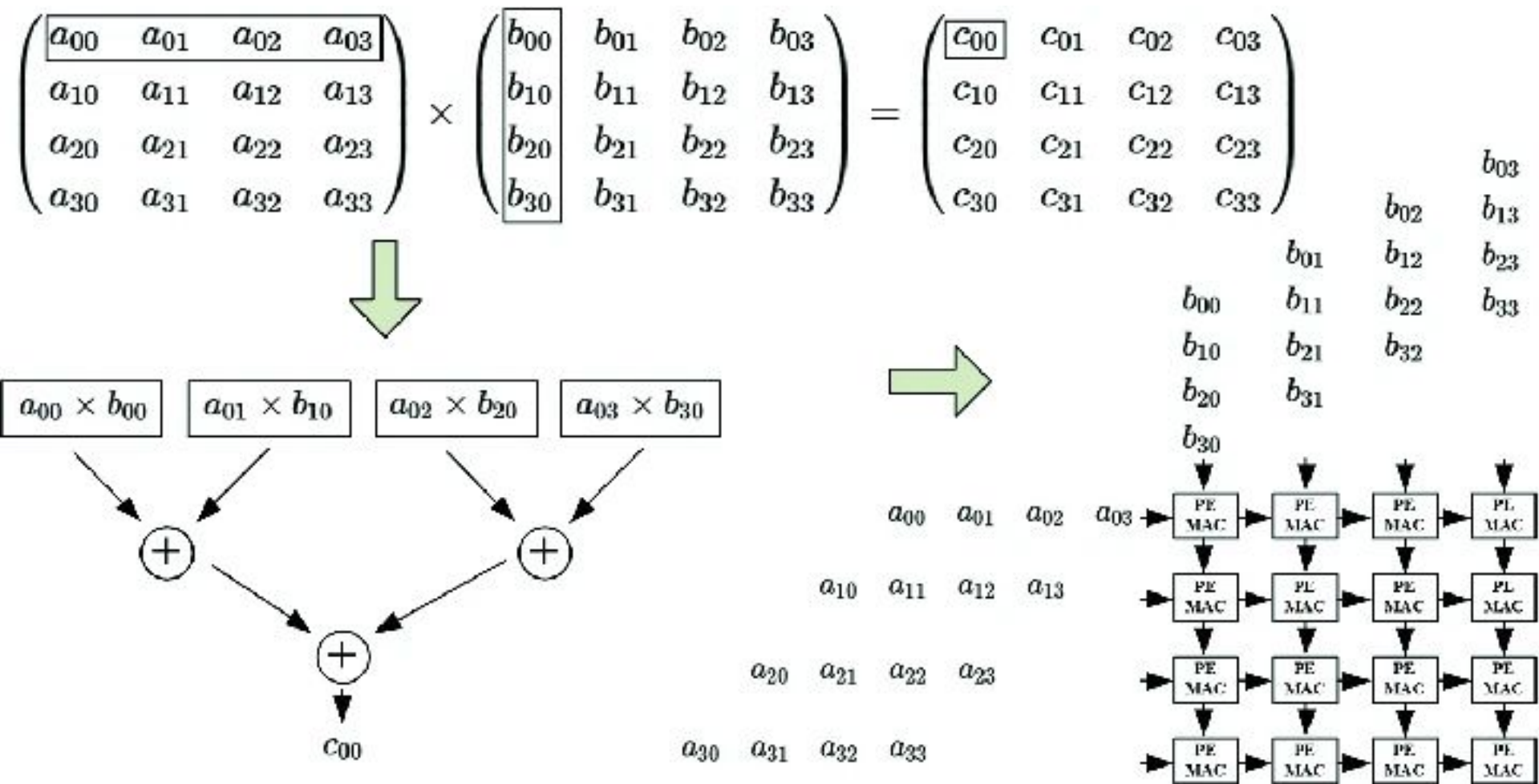
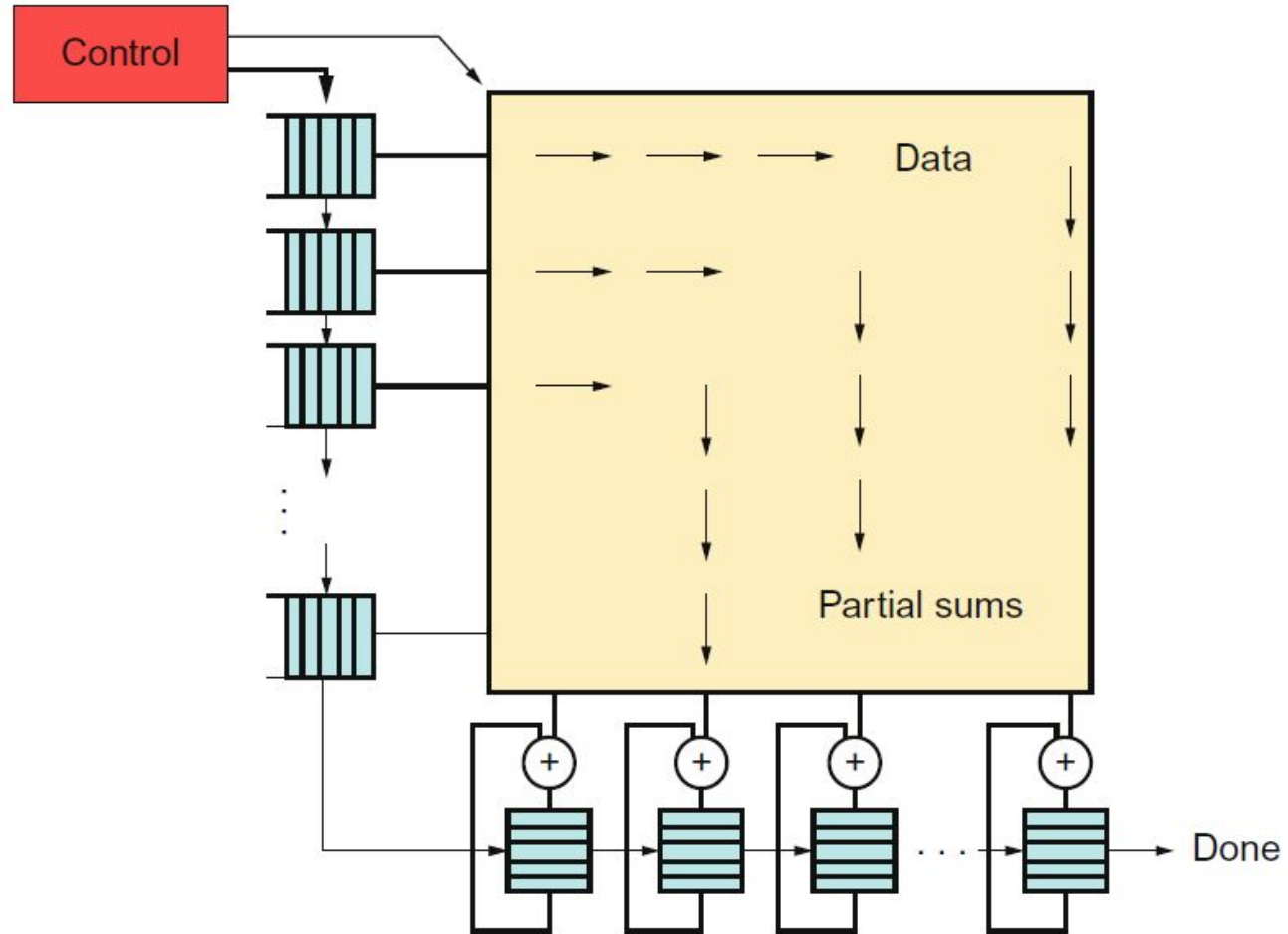


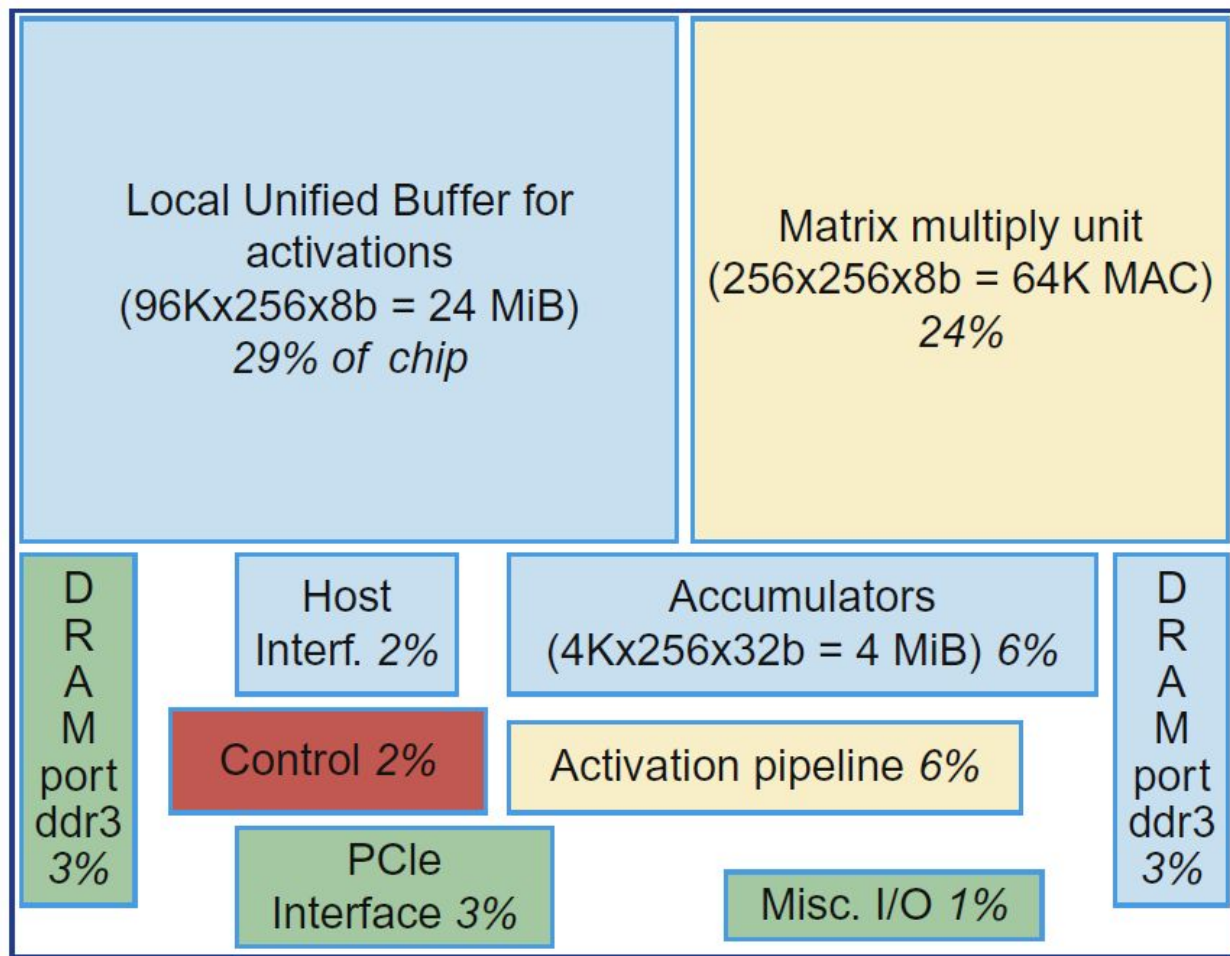
Fig: TPU Block Diagram. The PCIe bus is Gen3 16. The main computation part is the light-shaded Matrix Multiply Unit in the upper-right corner. Its inputs are the medium-shaded Weight FIFO and the medium-shaded Unified Buffer and its output is the medium-shaded Accumulators.



Systolic data flow of the Matrix Multiply Unit.

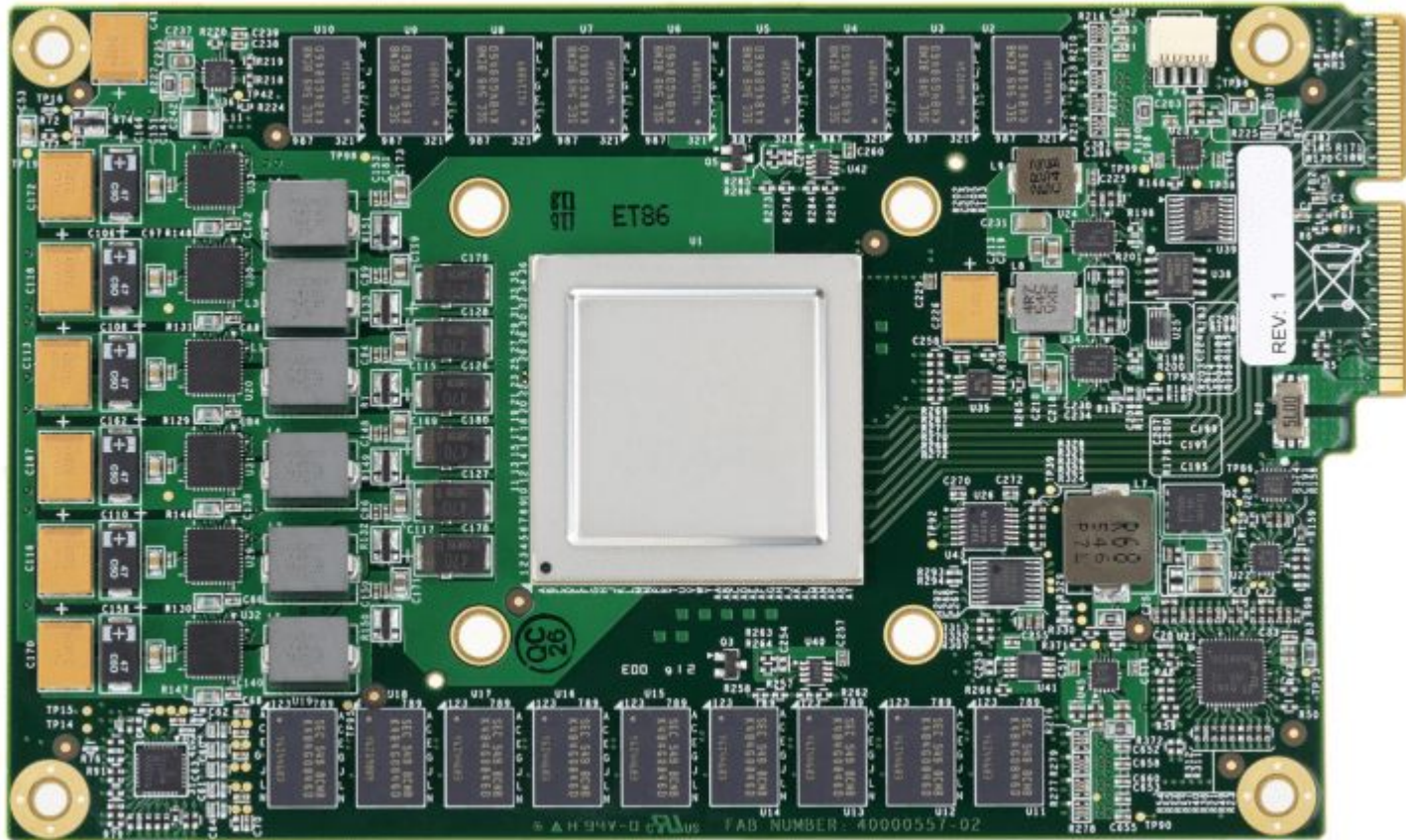


Floor plan of TPU die



TPU printed circuit board

TPU inserted
into the slot for
an SATA disk
in a server, but
the card uses
the PCIe bus.



```
# Create model
def multilayer_perceptron(x, weights, biases):
    # Hidden layer with ReLU activation
    layer_1 = tf.add(tf.matmul(x, weights['h1']), biases['b1'])
    layer_1 = tf.nn.relu(layer_1)
    # Hidden layer with ReLU activation
    layer_2 = tf.add(tf.matmul(layer_1, weights['h2']), biases['b2'])
    layer_2 = tf.nn.relu(layer_2)
    # Output layer with linear activation
    out_layer = tf.matmul(layer_2, weights['out']) + biases['out']
    return out_layer
```

```
# Store layers weight & bias
weights = {
    'h1': tf.Variable(tf.random_normal([n_input, n_hidden_1])),
    'h2': tf.Variable(tf.random_normal([n_hidden_1, n_hidden_2])),
    'out': tf.Variable(tf.random_normal([n_hidden_2, n_classes]))
}
biases = {
    'b1': tf.Variable(tf.random_normal([n_hidden_1])),
    'b2': tf.Variable(tf.random_normal([n_hidden_2])),
    'out': tf.Variable(tf.random_normal([n_classes]))
}
```